GitHub Copilot for ML/Data Ops

Beyond code completion

GitHub Copilot isn't just autocomplete for Python — it's a data engineering co-pilot that understands SQL dialects, PySpark transformations, ML pipelines, and your team's data conventions. The question isn't whether to use it — it's whether you're using 10% or 90% of what it can do.

- Copilot

What You'll Be Able to Do After This Session

- Generate SQL schemas, queries, and seed data using Copilot chat with context-aware prompts
- Build PySpark/pandas data pipelines from natural language descriptions in Jupyter notebooks
- Use Copilot to accelerate EDA, feature engineering, and hyperparameter tuning workflows
- Apply the mercury-data-engineering agent skill to auto-load data standards into every session
- Use Copilot CLI interactively for data pipeline debugging and SQL generation from your terminal

Copilot Surfaces for Data Engineers

🔧 Inline Chat (Ctrl+I)
└─ Use inside Jupyter cells: "Convert this pandas operation to PySpark"
└─ Inline SQL: "Add a window function for running average"

💬 Copilot Chat (Ctrl+Shift+I)
└─ "Explain why this Spark job is slow — here's the execution plan"
└─ "@workspace What data sources does the ETL pipeline use?"

🚀 Copilot Agent Mode
└─ Multi-file: Update schema changes across all downstream notebooks
└─ Refactor: Convert raw SQL strings to parameterised queries

📓 Notebook Integration
└─ Run cells directly from Copilot suggestions
└─ Get notebook summary for context
└─ Inline completions aware of previous cell outputs

Context Variables & Participants for Data Engineers

 # Variables (type # in chat):

`#file:schema.sql` → Include your schema for context-aware queries
`#file:etl_pipeline.py` → Reference existing pipeline code
`#selection` → Explain or transform selected DataFrame code
`#editor` → Analyse current notebook cell

@ Participants (type @ in chat):

`@codebase` → "Find all tables that reference the customers table"
`@terminal` → "Explain this Spark error in my terminal"

Context Variables & Participants for Data Engineers

🔍 **#web (GA):**

```
"#web What's the latest PySpark API for reading Delta Lake tables?"
```

Pro Tip for Data Engineers:

Always include your schema file when generating SQL:
`"#file:schema.sql Write a query to find customers with no orders in the last 90 days"`
Result: Copilot knows your actual table/column names — no hallucinated columns!

ML/Data Ops Use Cases

Schema Generation

Generate table **schemas** with relevant foreign key constraints

```sql
drop table if exists dbo.comments
drop table if exists dbo.likes
drop table if exists dbo.posts
drop table if exists dbo.users

create table dbo.users (
    id int identity(1,1) not null,
    name varchar(50) not null,
    primary key (id)
);

create table dbo.posts (
    id int identity(1,1) not null,
    title varchar(50) not null,
    content varchar(50) not null,
    user_id int not null,
    primary key (id),
    foreign key (user_id) references dbo.users(id)
);

create table dbo.comments (
    id int identity(1,1) not null,
    content varchar(50) not null,
    user_id int not null,
    post_id int not null,
    primary key (id),
    foreign key (user_id) references dbo.users(id),
    foreign key (post_id) references dbo.posts(id)
);

create table dbo.likes (
    id int identity(1,1) not null,
    user_id int not null,
    post_id int not null,
    primary key (id),
    foreign key (user_id) references dbo.users(id),
    foreign key (post_id) references dbo.posts(id)
);
```

Schema Generation

Basic schema generation:
Prompt: "Generate a SQL Server schema for a customer order system with:
- customers (id, name, email, created_date)
- orders (id, customer_id, total, status, order_date)
- order_items (id, order_id, product_id, quantity, unit_price)
Include primary keys, foreign keys, NOT NULL constraints, and CHECK constraints for status"

Schema from existing data:
Prompt: "#file:customers.csv Infer a SQL schema from this CSV file.
Choose appropriate data types, add constraints, and create indexes for columns likely to be queried frequently"

Schema evolution:
Prompt: "#file:schema.sql Add a 'loyalty_tier' column to customers with values (bronze, silver, gold, platinum) and a computed column for total_lifetime_spend based on the orders table"

Data Generation

Generate dummy **data** for development and testing

```sql
-- insert 100 random data into all existing tables in the database (users, posts, comments and likes) using loops
-- Language: sql
declare @i int = 1
declare @j int = 1
declare @k int = 1
declare @l int = 1

while @i <= 100
begin
    insert into dbo.users (name) values ('user' + cast(@i as varchar(50)))
    set @i = @i + 1
end

while @j <= 100
begin
    insert into dbo.posts (title, content, user_id) values ('title' + cast(@j as varchar(50)), 'content' + cast(@j as varchar(50)), @j)
    set @j = @j + 1
end

while @k <= 100
begin
    insert into dbo.comments (content, user_id, post_id) values ('comment' + cast(@k as varchar(50)), @k, @k)
    set @k = @k + 1
end

while @l <= 100
begin
    insert into dbo.likes (user_id, post_id) values (@l, @l)
    set @l = @l + 1
end;
```

Azure Data Studio

Data Generation

Realistic test data with pandas:
Prompt: "Generate a Python script using Faker library to create 10,000 rows of realistic customer data with:
- UK-formatted names, addresses, phone numbers
- Realistic order patterns (Pareto distribution — 20% of customers make 80% of orders)
- Date range: last 2 years, with seasonal patterns
- Output as both CSV and Parquet formats"

SQL seed data:
Prompt: "#file:schema.sql Generate INSERT statements for 100 rows of test data that satisfies all foreign key constraints and includes edge cases:
- NULL optional fields (10% of rows)
- Maximum length strings
- Boundary dates (first/last day of month)"

PySpark synthetic data:
Prompt: "Generate a PySpark script to create a synthetic dataset of 1M rows simulating IoT sensor readings with:
- 50 unique sensors, reading every 5 minutes
- Realistic temperature values (15-35°C with anomalies at 0.1% rate)
- Occasional NULL readings (sensor offline at 0.5% rate)
- Write to Delta Lake format partitioned by date"

SQL Query Generation

Generate SQL queries based on context and/or comments

```sql
select p.id, p.title, p.content, count(l.id) as likes_count
from dbo.posts p
left join dbo.likes l on p.id = l.post_id
```

```sql
-- show all posts order by likes count
-- Language: sql
select p.id, p.title, p.content, count(l.id) as likes_count
from dbo.posts p
left join dbo.likes l on p.id = l.post_id
group by p.id, p.title, p.content
order by likes_count desc

-- show all posts order by likes count using ctes
-- Language: sql
with cte as (
    select p.id, p.title, p.content, count(l.id) as likes_count
    from dbo.posts p
    left join dbo.likes l on p.id = l.post_id
    group by p.id, p.title, p.content
)
select * from cte order by likes_count desc
```

Azure Data Studio

SQL Query Generation

Context-aware query generation:
Prompt: "#file:schema.sql Write a query to find customers who made their first purchase in the last 30 days but haven't ordered since. Include customer name, first order date, and days since last activity."

Complex analytics:
Prompt: "#file:schema.sql Write a CTE-based query that calculates:
1. Monthly revenue trend (last 12 months)
2. Month-over-month growth percentage
3. Running 3-month average
4. Flag months where growth < -10% as 'alert'
Use window functions, not subqueries."

Performance-aware:
Prompt: "Rewrite this query to avoid the table scan. The orders table has 50M rows with indexes on (customer_id), (order_date), and (status, order_date).
[paste query]"

Library Imports

Auto import required libraries from initial input

```
# The model will use the Iris dataset to predict the species of an iris
# using a logistic regression classifier.

# Importing the necessary libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn import datasets
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix
```

Library Imports

Intelligent import generation:

Prompt: "I'm building a data pipeline that reads CSV files, transforms with pandas, trains a Random Forest classifier, evaluates with cross-validation, and saves the model. Generate all required imports organised by category (data, ML, evaluation, serialisation)."

Expected output:

```python
# Data processing
import pandas as pd
import numpy as np
# Machine Learning
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import LabelEncoder, StandardScaler
# Evaluation
from sklearn.metrics import classification_report, confusion_matrix
# Serialisation
import joblib
```

Comment Driven Development

Generate detailed code based on comments

```python
# Let's visualize the distribution of the penguins species with a bar plot in matplotlib
import matplotlib.pyplot as plt
plt.style.use('seaborn-whitegrid')
fig = plt.figure(figsize=(10, 6))
ax = plt.axes()
ax.bar(df['Species'].unique(), df['Species'].value_counts())
ax.set_title('Number of penguins by species')
ax.set_xlabel('Species')
ax.set_ylabel('Number of penguins')
plt.show()

```

```python
# Let's visualize with boxplots how the FlipperLength, CulmenLength and CulmenDepth are distributed for each species
# importing seaborn

import seaborn as sns
sns.set_style("whitegrid")
fig, ax = plt.subplots(1, 3, figsize=(20, 6))
sns.boxplot(x='Species', y='FlipperLength', data=df, ax=ax[0])
sns.boxplot(x='Species', y='CulmenLength', data=df, ax=ax[1])
sns.boxplot(x='Species', y='CulmenDepth', data=df, ax=ax[2])
plt.show()


✓ 0.4s
```

```python

# Split the data into training and test sets in a way to have 30% of the data for testing

from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.30, random_state=0)

```

DevOps

Use Copilot to generate your CI/CD deployment pipelines

GitHub Copilot

Used 1 reference

Here is the YAML code to create a GitHub Action workflow to train an Azure Machine Learning model based on a training script called `train.py`. This workflow is triggered on a push to the master branch.

```yaml
name: Azure ML Workflow

on:
  push:
    branches:
      - master

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
    - uses: actions/checkout@v2
    - uses: azure/login@v1
      with:
        creds: ${{ secrets.AZURE_CREDENTIALS }}
    - name: Azure ML Train
      uses: Azure/aml-run@v1
      with:
        azure_credentials: ${{ secrets.AZURE_CREDENTIALS }}
        workspace_name: 'your-azure-ml-workspace-name'
        compute_target: 'your-azure-ml-compute-target'
        experiment_name: 'your-experiment-name'
        script: 'train.py'
```

Please replace `'your-azure-ml-workspace-name'`, `'your-azure-ml-compute-target'`, and `'your-experiment-name'` with your actual Azure ML workspace name, compute target, and experiment name respectively.

Also, you need to set up `AZURE_CREDENTIALS` secret in your GitHub repository. The secret should contain the JSON output of the Azure service principal creation command.

DevOps

Prompt: "Generate a Dockerfile for a Python data pipeline that:
- Uses Python 3.11-slim base image
- Installs PySpark 3.5, pandas, scikit-learn, delta-spark
- Copies the pipeline scripts and config
- Runs as non-root user 'dataeng'
- Sets SPARK_HOME and JAVA_HOME environment variables
- Health check: verify spark-submit is accessible“

Prompt: "Generate a GitHub Actions workflow for a data pipeline that:
- Triggers on push to main and on schedule (daily at 02:00 UTC)
- Runs data quality checks with Great Expectations
- Executes the ETL pipeline
- Publishes data quality report as artifact
- Sends Slack notification on failure
- Uses Azure OIDC for authentication to ADLS"

Feature Engineering

Can **generate features** based on the columns and their relationships with each other

```python
# feature engineering
# Create a new feature called CulmenRatio
df['CulmenRatio'] = df['CulmenLength'] / df['CulmenDepth']

# Create a new feature called FlipperRatio
df['FlipperRatio'] = df['FlipperLength'] / df['BodyMass']

# Create a new feature called BodyMassRatio
df['BodyMassRatio'] = df['BodyMass'] / df['CulmenDepth']

# Create a new feature called CulmenVolume
df['CulmenVolume'] = df['CulmenLength'] * df['CulmenDepth'] * df['CulmenDepth']

# Create a new feature called FlipperVolume
df['FlipperVolume'] = df['FlipperLength'] * df['FlipperLength'] * df['FlipperLength']

# Create a new feature called BodyMassVolume
df['BodyMassVolume'] = df['BodyMass'] * df['BodyMass'] * df['BodyMass']

# Create a new feature called BodyMassCulmenRatio
df['BodyMassCulmenRatio'] = df['BodyMass'] / df['CulmenLength']
```

Hyperparameter tuning

Copilot can be used to help generate code for **hyperparameter tuning**

```python
# This file is used for hyperparameter turning using bayesian optimisation of a logistic regression model

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score
from sklearn.metrics import roc_auc_score
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline

from bayes_opt import BayesianOptimization
from bayes_opt import UtilityFunction

# Load data
X = pd.read_csv('X.csv')
y = pd.read_csv('y.csv')

# Split data into training and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

# Define the model
def logistic_regression(C, penalty):
    model = Pipeline([
        ('scaler', StandardScaler()),
        ('classifier', LogisticRegression(C=C, penalty=penalty))
    ])
    return model

# Define the function to optimise

def function_to_optimise(C, penalty):
    model = logistic_regression(C, penalty)
    return cross_val_score(model, X_train, y_train, scoring='roc_auc', cv=5).mean()

# Define the bounds of the hyperparameters
pbounds = {'C': (0.01, 100), 'penalty': ('l1', 'l2')}
optimizer = BayesianOptimization(
    f=function_to_optimise,
    pbounds=pbounds,
    random_state=1,
)

# Optimise the model
optimizer.maximize(
    init_points=10,
    n_iter=100,
)

# Print the best hyperparameters
print(optimizer.max)
```

Parallel Processing Optimisation

Optimisation of code using parallel processing can be achieved

can you optimise this for parallel processing

GitHub Copilot

Used 1 reference

Yes, the `cross_val_score` function in scikit-learn can be optimized for parallel processing by setting the `n_jobs` parameter to `-1`. This will use all processors available to compute the cross-validation scores in parallel, which can significantly speed up the process if you have a multi-core processor.

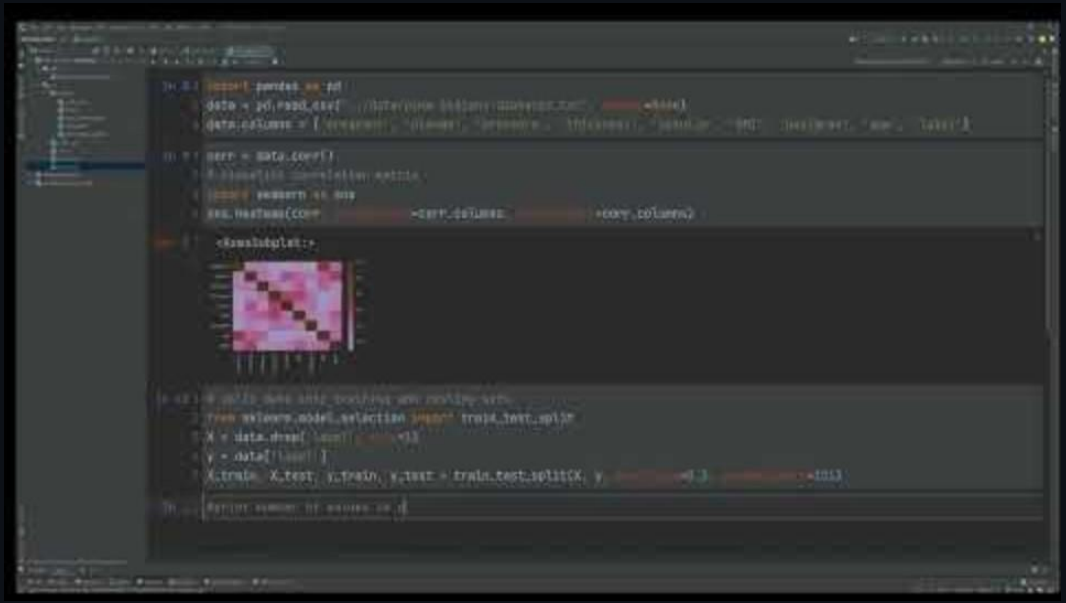
Here's how you can modify the `function_to_optimise` function to enable parallel processing:

```
def function_to_optimise(C, penalty):
    model = logistic_regression(C, penalty)
    return cross_val_score(model, X_train, y_train, scoring='roc_auc', cv=5, n_jobs=-1).mean()
```

Please note that parallel processing can increase the memory usage, as each process will load a copy of the data. Make sure your machine has enough memory to handle the increased load.

EDA (Exploratory Data Analysis)

Analyse a dataset and rapidly develop a basic model



Understand complex code

Use of /explain can assist in making code easier to understand

@workspace /explain this file

GitHub Copilot used @workspace

Used 1 reference

This Python script is used for hyperparameter tuning of a Logistic Regression model using Bayesian Optimization. Here's a brief overview of what each section does:

1. **Imports**: The necessary libraries and modules are imported. These include numpy, pandas, matplotlib, sklearn, and bayes_opt.
2. **Data Loading**: The predictor variables (X) and target variable (y) are loaded from 'X.csv' and 'y.csv' respectively.
3. **Data Splitting**: The data is split into training and test sets, with 80% of the data used for training and 20% used for testing.
4. **Model Definition**: A function `logistic_regression` is defined to create a Logistic Regression model with given hyperparameters 'C' and 'penalty'. The model is part of a pipeline that first scales the data using StandardScaler.
5. **Optimization Function Definition**: A function `function_to_optimise` is defined to return the mean ROC AUC score from 5-fold cross-validation of the Logistic Regression model on the training data.
6. **Hyperparameter Bounds Definition**: The bounds for the hyperparameters 'C' and 'penalty' are defined in `pbounds`.
7. **Bayesian Optimization**: A BayesianOptimization object is created with the function to optimize and the hyperparameter bounds. The optimization is performed with 10 initial points and 100 iterations.
8. **Best Hyperparameters**: The best hyperparameters found by the optimization are printed.

The cursor is currently on line 26, which is part of the `logistic_regression` function definition.

Unit Testing Generation - Copilot Chat

Works best for general helper functions with well defined input and output

CHAT: GITHUB COPILOT

bryantson

Can you help me to generate 3 unit testing methods for different possible case scenarios for all functions in selected Puzzle.java?

GitHub Copilot

Sure, I can help you with that. Here are three unit testing methods for each function in the Puzzle.java file:

```java
import org.junit.jupiter.api.Assertions;
import org.junit.jupiter.api.Test;

public class PuzzleTest {

    @Test
    public void testSumEverythingOddNumber() {
        Puzzle puzzle = new Puzzle();
        int[] list = {1, 2, 3, 4, 5};
        int expectedSum = 9;
        int actualSum = puzzle.sum_everything_odd_number(list);
        Assertions.assertEquals(expectedSum, actualSum);
    }

    @Test
    public void testSumEverythingOddNumberWithEmptyList() {
        Puzzle puzzle = new Puzzle();
        int[] list = {};
        int expectedSum = 0;
        int actualSum = puzzle.sum_everything_odd_number(list);
        Assertions.assertEquals(expectedSum, actualSum);
    }

    @Test
    public void testConcatenateStrings() {
        Puzzle puzzle = new Puzzle();
        String first = "Hello";
        String second = "World";
        String expectedConcatenation = "HelloWorld";
        String actualConcatenation = puzzle.concatenateStrings(fi
        Assertions.assertEquals(expectedConcatenation, actualConc
    }

    @Test
    public void testConcatenateStringsWithEmptyStrings() {
        Puzzle puzzle = new Puzzle();
        String first = "";
        String second = "";
        String expectedConcatenation = "";
        String actualConcatenation = puzzle.concatenateStrings(fi
        Assertions.assertEquals(expectedConcatenation, actualConc
```

Can you explain how to run these unit tests in Visual Studio Code?

Ask Copilot or type '/' for commands

Untitled (Workspace)

Puzzle.java

JavaDemo > Puzzle.java

```java
// This code sums the numbers from 1 to 100 and prints the result.

public class Puzzle {

    private int sum_everything_odd_number(int[] list) { // sum_everything_od
        int sum = 0; // sum is an integer variable with the value 0
        for (int i = 0; i < list.length; i++) { // for loop that iterates th
            if (list[i] % 2 == 1) { // if the number is odd
                sum += list[i]; // add the number to the sum
            }
        }
        return sum; // return the sum
    }

    private String concatenateStrings(String first, String second) {
        return first + second;
    }

    public static void main(String[] args) {
        Puzzle puzzle = new Puzzle();
        int[] list = {1, 2, 3, 4, 5};
        System.out.println(puzzle.sum_everything_odd_number(list));
        System.out.println(puzzle.concatenateStrings("Hello", "World"));
    }

}
```

Copilot + Jupyter Notebooks: The Data Engineer's Workflow

📓 Notebook-Aware Completions
└─ Copilot sees ALL cells above the current cell
└─ Variables, imports, and DataFrames from previous cells are in context
└─ Tip: Run cells in order so Copilot understands your data shape

🔧 Inline Chat in Cells (Ctrl+I)
└─ "Convert this cell from pandas to PySpark"
└─ "Add error handling to this data loading cell"
└─ "Visualise the result of the previous cell as a bar chart"

💬 Copilot Chat for Notebooks
└─ "#file:analysis.ipynb Summarise what this notebook does"
└─ "@codebase Which notebooks use the customers table?"
└─ "The last cell threw a KeyError — explain why and fix it"

📔 Notebook Generation
└─ "/newNotebook Create a Jupyter notebook for penguin species classification"
└─ Copilot generates cells with markdown headers + code

Copilot + Jupyter Notebooks: The Data Engineer's Workflow

```
✅ Best Practices:
  1. Keep a "Setup" cell with all imports at the top
  2. Use markdown cells between code cells (Copilot reads them for context)
  3. Name DataFrames descriptively (df_customers not df1)
  4. Run cells sequentially — Copilot's context follows execution order
  5. Start a new chat thread per analysis section
```

Data Validation with Copilot

Data quality is the biggest real-world pain point for data engineers

```
Great Expectations suite:
Prompt: "#file:customers.csv Generate a Great
Expectations validation suite for
this dataset that checks:
- customer_id is unique and never null
- email matches a valid email regex pattern
- created_date is within the last 5 years
- country is one of [UK, US, DE, FR, NL]
- age is between 18 and 120
- No duplicate rows
Generate as a Python script using the GX
library."
```

Data Validation with Copilot

Pandas data contract:
Prompt: "Generate a Pandera schema definition for this DataFrame that validates:
- Column types, nullable constraints, and value ranges
- Custom checks for business rules (e.g., end_date > start_date)
- Include informative error messages for each check"

Data quality dashboard:
Prompt: "Generate a data quality report function that:
- Calculates completeness (% non-null per column)
- Calculates uniqueness (% unique values per column)
- Identifies outliers using IQR method
- Returns a summary DataFrame and a matplotlib dashboard"

Agent Skills for GitHub Copilot

Agent Skills are discrete, task-specific instructions that automatically load when relevant to your current task. Think of them as invisible expertise that enhances Copilot's knowledge on-demand.

Key Characteristics:

- 🤫 **Silent - Auto-load based on context (no explicit invocation needed)**
- 📦 **Portable - Follow open standard (agentskills.io) - work across tools**
- 🎯 **Task-Specific - Focused instructions for particular scenarios**
- 🔄 **On-Demand - Only load when relevant to save context window**

Agent Skills - Task-Specific Instructions

What Is an Agent Skill? A skill is a folder containing:

- ✅ **SKILL.md file (required) - instructions and metadata**
- ✅ **Optional scripts (.py, .sh, .ps1)**
- ✅ **Optional templates (report.md, config.json)**
- ✅ **Optional reference files (examples, data)**

Directory Structure:

```
.github/skills/
  csv-analysis/              ← One skill
    SKILL.md                 ← Required
    scripts/
      analyze.py             ← Optional: supporting script
    templates/
      report-template.md     ← Optional: output template
  github-actions-debug/      ← Another skill
    SKILL.md
  mercury-unit-tests/        ← Mercury skill!
    SKILL.md
```

Mercury Data Engineering Agent Skill

Agent Skills for Data Engineering — The mercury-data-engineering Skill

File: `.github/skills/mercury-data-engineering/SKILL.md`

Once created, every Copilot session for any SQL, Python, or notebook file automatically loads these standards. No more "Copilot doesn't know we use PySpark, not pandas" complaints.

Mercury Data Engineering Agent Skill

```
---
name: mercury-data-engineering
description: >
  Auto-load Mercury data engineering standards: SQL conventions, PySpark patterns,
  data validation rules, notebook structure, and ML pipeline templates.
  Use when working on SQL files, Jupyter notebooks, Python data scripts,
  Spark jobs, or ML training pipelines.
---

## Mercury Data Engineering Standards

### SQL Conventions
- Schema prefix: always use `dbo.` for SQL Server, `public.` for PostgreSQL
- Naming: snake_case for tables and columns (e.g., `customer_orders`)
- Always include: PRIMARY KEY, NOT NULL on required columns, CHECK constraints
- Date columns: use `_date` suffix (e.g., `created_date`, `modified_date`)
- Indexes: create on all foreign key columns and frequently filtered columns
```

Mercury Data Engineering Agent Skill

```
### Python / PySpark Standards
- Python version: 3.11+
- PySpark version: 3.5+
- Use PySpark DataFrame API (not RDD) for all distributed operations
- Use pandas only for local analysis (< 1GB datasets)
- Always specify schema when reading CSV/JSON (don't rely on inference)
- Partition strategy: date-based partitioning for time-series data

### Notebook Standards
- Cell 1: Imports (organised by category)
- Cell 2: Configuration / Spark session setup
- Cell 3+: Analysis with markdown headers between sections
- Last cell: Cleanup (stop Spark session, close connections)
- Name DataFrames descriptively: df_customers, df_orders (not df, df2)

### Data Validation
- All pipelines must include validation before write
- Use Great Expectations or Pandera for schema validation
- Log validation results to Azure Log Analytics
- Pipeline must FAIL on critical validation errors (nulls in PK columns)
```

Copilot CLI for Data Engineers

GitHub Copilot CLI — Agentic Data Engineering from Your Terminal

Installation:

```
# Windows (WinGet)
winget install GitHub.CopilotCLI

# Cross-platform (npm)
npm install -g @github/copilot
```

Copilot CLI — Your Command Line Co-Pilot

Mode	How to start	Best for
Interactive	Type copilot	Iterative tasks, debugging pipelines, multi-step infra work
Programmatic	copilot -p "your prompt"	Scripted automation, CI/CD integration

Copilot CLI — Your Command Line Co-Pilot

```
# Start interactive session from your data project
copilot

# Scenario 1: SQL generation from natural language
"Generate a SQL query that finds all customers who churned
(no orders in 90 days) but were previously in the top 10%
by spend. Use the schema from schema.sql in this directory."

# Scenario 2: Convert a notebook to a production script
"Convert ml-penguin-demo.ipynb into a production-ready Python
script with argparse, logging, error handling, and MLflow
experiment tracking. Remove all EDA/visualisation code."
```

MCP Servers — Give Copilot Live Access to Your Data

What is MCP?

- A protocol that lets Copilot connect to external tools and data sources
- Copilot can read live schemas, run queries, and access real metadata

MCP Servers — Give Copilot Live Access to Your Data

Data Engineering MCP Servers:

🗄️ Database Servers (SQL Server, PostgreSQL, MySQL)
└─ Copilot reads your actual schema — no more hallucinated columns!
└─ Generate queries against real tables with correct column names
└─ "Write a query for the top 10 customers" — Copilot knows your schema

📊 Azure Data Explorer (KQL)
└─ Write KQL queries with schema awareness
└─ Copilot knows your table names and column types

📁 Azure Data Lake / Blob Storage
└─ List and describe files in your data lake
└─ Generate PySpark read commands with correct paths

MCP Servers — Give Copilot Live Access to Your Data

```
Setup Example (.vscode/mcp.json):
{
  "servers": {
    "mercury-sqlserver": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-mssql",
               "--connection-string", "${env:MERCURY_SQL_CONN}"]
    }
  }
}

Pro Tip: Never put connection strings in the config file!
Use environment variables or Azure Key Vault references.
```

CHAT

Show me the relationships between Catalogues and CatalogueBids tables

Untitled-1:1

GitHub Copilot

I'll help you understand the relationships between the Catalogues and CatalogueBids tables. Let me first examine the structure of both tables to identify their columns and relationships.

Ran `describe_table` – MSSQL MCP (MCP Server) ✓

Ran `describe_table` – MSSQL MCP (MCP Server) ✓

Now let me query some sample data to better understand the relationship:

Ran `read_data` – MSSQL MCP (MCP Server) ✓

Let me also get some statistics about the relationship:

Ran `read_data` – MSSQL MCP (MCP Server) ✓

Now let me create a visual representation of the relationship and write it to your untitled file:

Untitled-1

I've analyzed the relationship between the Catalogues and CatalogueBids tables and created a comprehensive overview in your file. Here are the key findings:

Relationship Summary:

- **Type**: One-to-Many (1:M)
- **Connection**: `CatalogueBids.CatalogueId` → `Catalogues.Id`
- **Meaning**: Each catalogue item can receive multiple bids

1 file changed

Untitled-1

Add Context... | settings.json Current file

Edit files in your workspace in agent mode

Agent | Claude Sonnet 4

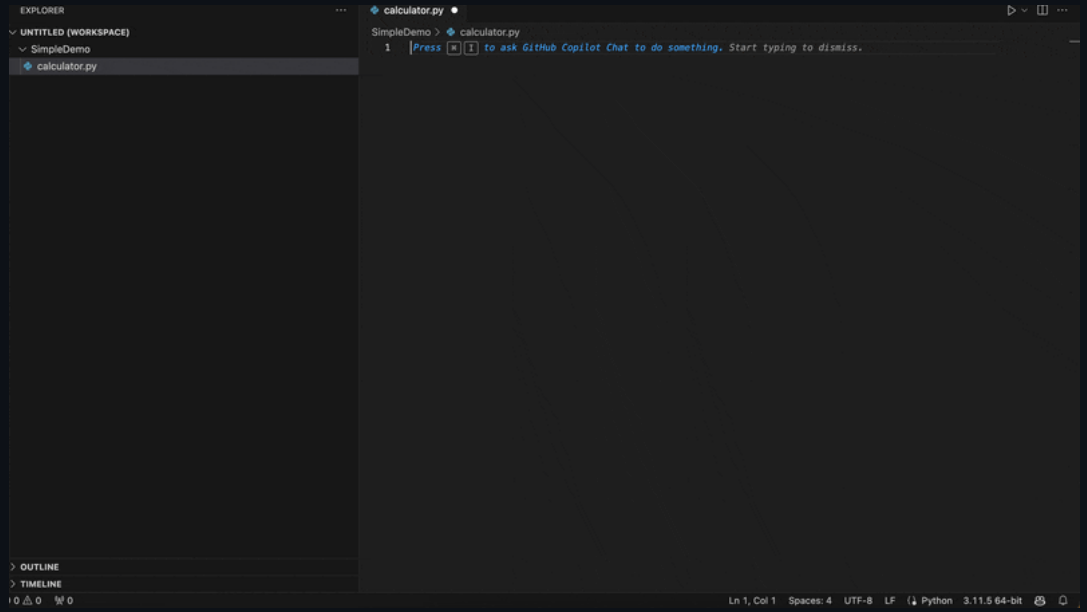
Common Problems

Problems #1:

Copilot fails to produce answer or keep repeating

Some problems

- Fails to produce answer
- Hallucination - Keeps repeating

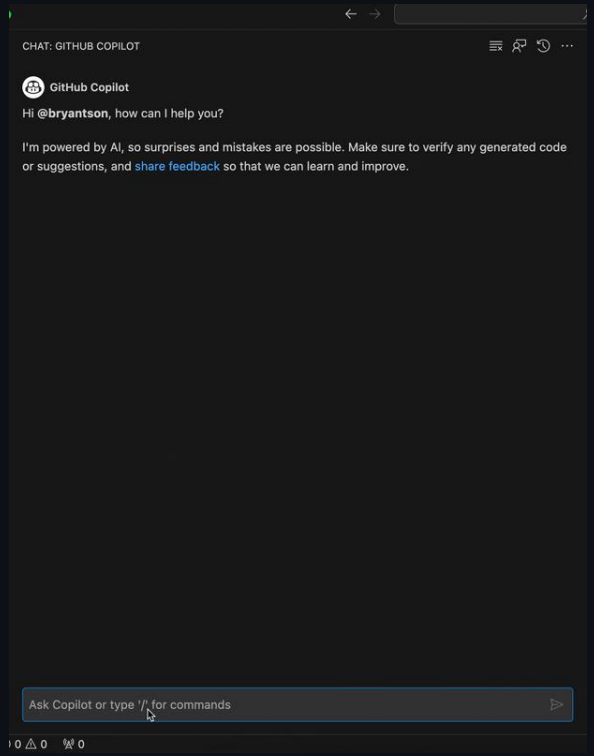


Problems #2:

Copilot generates incorrect result

Reasons why...

- Not enough context
- Old trained data
- Generative in nature

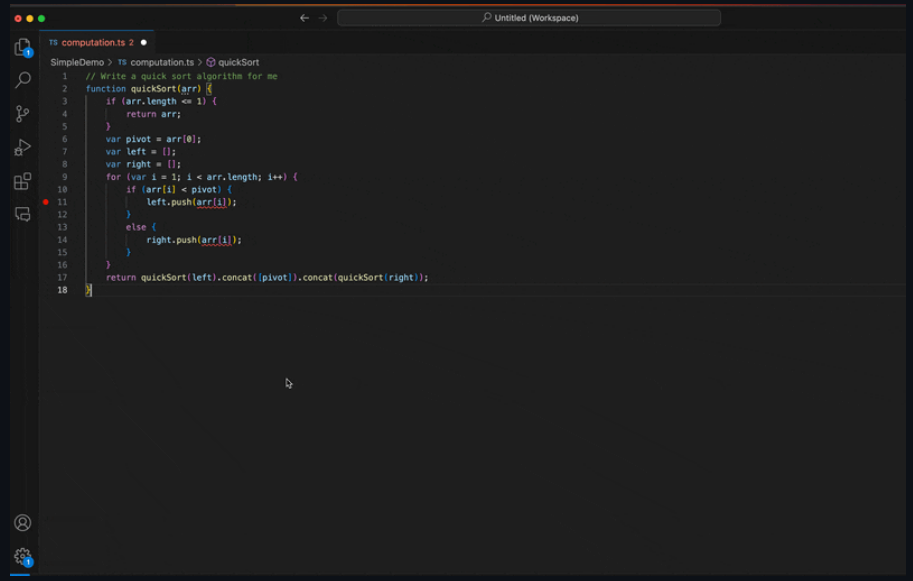


Problem #3:

Copilot suggests non-optimal solution

How to improve

- Ask in Copilot Chat
- Use in-line suggestion from Copilot



Lab

Frequently Asked Questions

“Is the information shared with GitHub Copilot secure?

“For example, let’s say I enter some sensitive information through GitHub Copilot, is it going to store the data like my user information?”

“Can I train GitHub Copilot on my private codebase?”

“Is the Copilot suggestion from an IDE based on my computer, a project opened in my IDE, files opened through tabs or just a single file?”

“What telemetry data can I get from GitHub Copilot ?”

“How can GitHub Copilot integrate with existing pipelines/automations/scripts that we have?

“For example, I have this GitHub Actions/Azure DevOps pipeline, can GitHub Copilot integrate with those products so our code will always be secure?”

“Will Copilot replace my job”

“How much of my work can it take over?”

Q & A

Leveraging GitHub Copilot for Data Engineering

- Boosting data engineering workflows
- https://github.com/CleveritDemo/copilot-data-engineering/tree/main
- What you'll practice:
  - RDD operations with PySpark
  - DataFrame transformations
  - SQL query generation
  - Data profiling and cleaning
- Prerequisites: Python 3.11+, VS Code, Jupyter extension
- Estimated time: 30-45 minutes

Leveraging GitHub Copilot for DataOps and Machine Learning

- Use GitHub Copilot to assist your DataOps and Machine Learning tasks.
- https://github.com/arinco-crew-community/copilot-ml-data-ops/tree/main
- What you'll practice:
  - SQL schema and data generation (sql-demo.sql)
  - Penguin species classification (ml-penguin-demo.ipynb)
  - ML Q&A for concept understanding (ml-q&a-demo.ipynb)
- Prerequisites: Python 3.11+, VS Code, Jupyter extension
- Estimated time: 45-60 minutes

Thank you